Axona Quick Start

Get a working pub/sub roundtrip in **under five minutes** on the current `@axona/protocol` **v4.11.2** API (kernel 4.11.2). One Node process connects to the live **testnet** bridge, subscribes to an open topic, publishes a signed message, and logs what comes back.

> **Testnet, for now.** The 4.x line runs on **testnet** (`wss://testnet.axona.net`). The production bridges (`wss://bridge.axona.net`) are still on the 3.x line, and the two don't interoperate (the wire major partitions them). So this Quick Start uses testnet + the `#v4.11.2` pin. To target production instead, install a `3.x` tag and point at `wss://bridge.axona.net`.

Companion documents:

- API Reference — every exported symbol.
- Programmer Guide — mental model + worked example.

Prerequisites

- **Node.js 20+** (for built-in Web Crypto Ed25519)
- A terminal with network access (we connect to `wss://testnet.axona.net`)

No build step, no DB, no local bridge — the testnet bridge is the entry point.

1. Install (30 seconds)

```
mkdir my-axona-demo && cd my-axona-demo
npm init -y
npm pkg set type=module
npm install github:axona-net/axona-protocol#v4.11.2
```

2. Two identities, one rule

Axona separates **who you connect as** from **who you publish as**:

- `createNodeIdentity({ lat, lng })` -> your **connection** identity. Its public key forms the 66-hex node ID (`.id`); the leading byte encodes your region. This authenticates the transport. It is fine to mint a fresh one each run.
- `createAuthorIdentity({ persistAs? })` -> your **author** identity. Its public key is the 64-hex Author ID (`.authorId`), which appears on the wire as `signerPubkey`. It has no location and no node ID — authorship is not a place. Pass `persistAs` to keep a stable author across runs.

The one rule: every publish names its signer via `{ signWith }` — an author identity, or the `ANONYMOUS` sentinel for a deliberately unsigned post. There is no default signer, and the node key never signs publishes.

3. Topics are descriptors, not strings

A topic is a small descriptor `{ region?, owner?, name, write? }`; its 66-hex topic ID is a hash of that descriptor. Because the ID is a pure function of the fields, **anyone who knows the fields computes the same ID** — no registry, no coordination.

- `region` — a real geographic cell. Accepts a name (`'useast'`), a hex string (`'0x89'`), or a number (`137`) — all equivalent. Omit it and it defaults to the publisher's own node region; name it to share a topic across regions.
- `owner` — an Author ID, or absent.
- `name` — the human label (`'lobby'`, `'profile'`).
- `write` — `'open'` or `'owner'`. **Defaults by owner:** no owner -> `'open'` (anyone publishes); an owner -> `'owner'` (only the owner key publishes, the safe default). With no owner, `write` is ignored — you can't have an owner-only topic with no owner.

This Quick Start uses an **open** topic `{ region, name }`: anyone publishes, anyone subscribes. (Note: the removed v2 model — string topics, a `publisher` argument, `sign:false`, `geoCellId/regionSynthPublisher` synthetic publishers — is gone. Topics are descriptors now.)

`deriveTopicId(descriptor)` returns the 66-hex ID — the shareable read handle. `sub` accepts **either** a descriptor **or** that ID; `pub` always needs the descriptor (the storing node recomputes the ID to enforce the write policy).

4. Write the demo (one file)

Save this as `index.js` — it mirrors how `apps/axona-minimal` wires a peer:

```
import {
  AxonaPeer, AxonaDomain, NeuronNode,
  createNodeIdentity, createAuthorIdentity,
  deriveTopicId, KERNEL_VERSION,
} from '@axona/protocol';
import { webTransport } from '@axona/protocol/transport/web/index.js';

const BRIDGE = 'wss://testnet.axona.net';           // live testnet bridge (kernel 4.11.2)
const HERE   = { lat: 38.0, lng: -77.0 };           // your real location (us-east here)
const TOPIC  = { region: 'useast', name: 'quick-start-demo' };   // open topic

// 1. Two identities: connection (node) + authorship (author).
const node$identity = await createNodeIdentity(HERE);            // ephemeral connection key
const author        = await createAuthorIdentity();             // ephemeral author key

// 2. Build the peer on the web transport pointed at the bridge.
const transport = webTransport({ bridgeUrl: BRIDGE, identity: node$identity });
const node      = new NeuronNode({ id: BigInt('0x' + node$identity.id), lat: HERE.lat, lng: HERE.lng })
node.transport  = transport;
const peer = new AxonaPeer({
  domain: new AxonaDomain({ k: 20 }),
  node,
  nodeIdentity: node$identity,
  transport,
});

await transport.start(node$identity.id);
await peer.start();

// 3. Wait for the routing mesh to warm up before pub/sub. peer.ready() does this
//    for you - it resolves once enough peers are in the synaptome (or a stable
//    non-zero plateau), bounded by timeoutMs. No hand-rolled polling loop needed.
console.log('kernel v' + KERNEL_VERSION + ' - connecting…');
const status = await peer.ready();                // { ready, peers, ms, reason }
console.log('mesh ' + (status.ready ? 'ready' : 'not ready') + ' (' + status.peers + ' peers, ' + statu
console.log('topic id:', await deriveTopicId(TOPIC));

// 4. Subscribe. The handler gets an envelope: { msgId, seq, ts, topic, message, signerPubkey? }.
//    It fires for every delivery - including the ones we publish below - so you
//    watch the counter climb live in the console.
const sub = await peer.sub(TOPIC, (env) => {
  console.log('[recv]', env.message, 'from', (env.signerPubkey || 'anon').slice(0, 12));
}, { since: 'all' });
```

```
// 5. Publish on a loop: one signed message per second, each carrying an
//    incrementing counter, so the system is visibly, continuously running.
let n = 0;
const timer = setInterval(async () => {
  const msg = `tick #${++n}`;
  const msgId = await peer.pub(TOPIC, msg, { signWith: author });
  console.log('[pub ]', msg, '(msgId ' + msgId.slice(0, 12) + '…)');
}, 1000);

// 6. Keep running until Ctrl+C, then leave cleanly.
process.on('SIGINT', async () => {
  clearInterval(timer);
  await sub.stop();
  await peer.leave();
  process.exit(0);
});
```

5. Run

```
node index.js
```

You should see the counter climb, one tick per second, until you stop it with **Ctrl+C**:

```
kernel v4.11.2 - connecting…
mesh ready (4 peers, 1200ms)
topic id: 89a1b2c3…
[pub ] tick #1 (msgId 8e9d4b1a30c2…)
[recv] tick #1 from 4f2a9b0c1d3e
[pub ] tick #2 (msgId 2c7f0a9b41d3…)
[recv] tick #2 from 4f2a9b0c1d3e
[pub ] tick #3 (msgId 91b3e5c8007a…)
[recv] tick #3 from 4f2a9b0c1d3e
…
```

That's it. You just:

- minted an Ed25519 connection identity (66-hex node ID, region-prefixed) and a location-free author identity (64-hex Author ID = `signerPubkey`),
- connected to the live bridge over the web transport and warmed up a routing mesh,
- derived a deterministic topic ID from `{ region, name }`,
- published a **signed** envelope to the topic's root axons and received it back through a `since: 'all'` subscriber.

What just happened

```
peer.pub({region,name}, msg, {signWith: author})        peer.sub({region,name}, handler, {since:'all
     |                                                       |
     v                                                       v
deriveTopicId({region, name})  ----- same fields ----->  deriveTopicId({region, name})
     |   (identical 66-hex topic id on both sides)           |
     v                                                       v
build signed envelope  -> route to topic's K-closest    subscribe-k to the same K-closest root axons
root axons (publish-k)                                       |
     |                                                       v
     +--------------------- fan-out ----------------->  handler(env): { msgId, seq, ts, topic, messag
```

Both sides compute the identical topic ID because the ID is a pure hash of the descriptor fields. That ID-matching is the rule you can't break — same `{ region, name }`, same topic.

Going further

Want to…	Do this
Make authorship stable across runs	`createAuthorIdentity({ persistAs: 'me' })`
Publish anonymously	`peer.pub(topic, msg, { signWith: ANONYMOUS })` (import `ANONYMOUS`)
Own a feed only you can write	`{ region, owner: me.authorId, name: 'profile' }` (write defaults to `'owner'`)
Share a read-only handle	`await deriveTopicId(descriptor)` -> hand out the 66-hex ID; `sub`/`pull`/`metrics` accept it
Run against production (3.x)	install a `3.x` tag and set `BRIDGE = 'wss://bridge.axona.net'` (prod is a separate, non-interoperating line)
See the full mental model	Programmer Guide
Look up a specific symbol	API Reference

Troubleshooting

`Cannot find module '@axona/protocol'` — make sure `package.json` has `"type": "module"` and the install completed. Re-run `npm install`.

`Cannot mix BigInt and other types` — `NeuronNode` XORs node IDs as BigInts. Pass `BigInt('0x' + node$identity.id)`, not the raw hex string.

`peer.pub: name a signer…` — `signWith` is required. Pass an author identity, or `{ signWith: ANONYMOUS }` for an unsigned publish. There is no default signer.

Connects but nothing arrives — you almost certainly published before the mesh warmed up. Subscribing/publishing on a cold synaptome routes to too few root axons. `await peer.ready()` (step 3) before calling `pub`/`sub` — it resolves once the routing mesh is warm. On the public bridge this takes a few seconds.

Can't reach the bridge / connection hangs — confirm outbound `wss://` (TLS WebSocket) to `testnet.axona.net` is allowed by your network. There is no fallback to a local process; the demo needs the bridge to find peers.

`UPGRADE_REQUIRED` close code (4426) — a wire/version mismatch. The testnet bridge runs kernel 4.11.2 (wire 4.0); install `github:axona-net/axona-protocol#v4.11.2`. Note this also fires if you point a 4.x peer at a **production** (3.x) bridge — they're a hermetic wire partition, so match the pin to the network.